Agenda:

1. Introduction of Queues
2. Queue Implementation via Dynamic Arrays
3. Queue Implementation via Linked List
4. Implement Queue via Stacks
5. N integers containing 1, 2 & 3

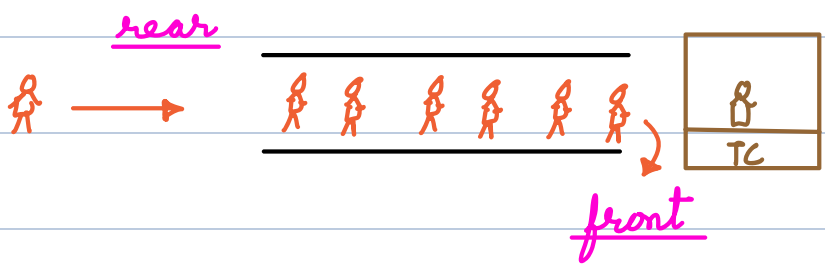
FIFO → First In First Out

Introduction of Queues

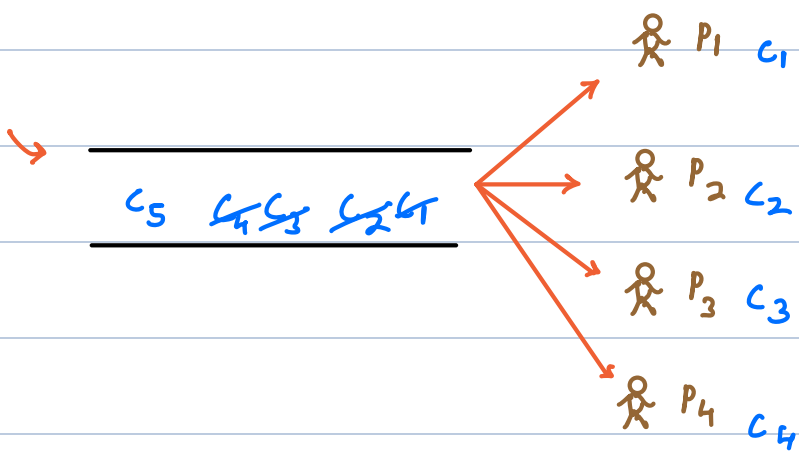
A queue represents a **linear data structure** where items are added at one end and removed from the other end. It follows the "**First-In-First-Out**" (**FIFO**) principle, meaning that the item that has been inserted first in queue will be the first one to be removed.

Examples:

- **Ticket Counter Queue**: Imagine you're at a ticket counter for a popular event, and there are several people waiting in line to purchase tickets. This line of people forms a queue, and the first person who has come in this line will get the ticket first.



- **Call Centre**



Operations of Queue:

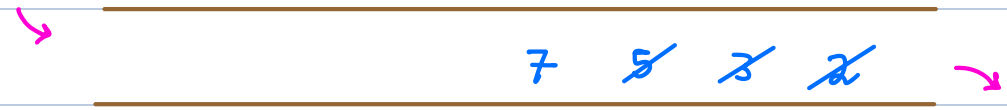
1. **Enqueue(x)** → insert x from rear end
2. **Dequeue( )** → remove element at front end
3. **front( )** → get element at front end
4. **rear( )** → get element at rear end
5. **isEmpty( )** → check if queue is empty

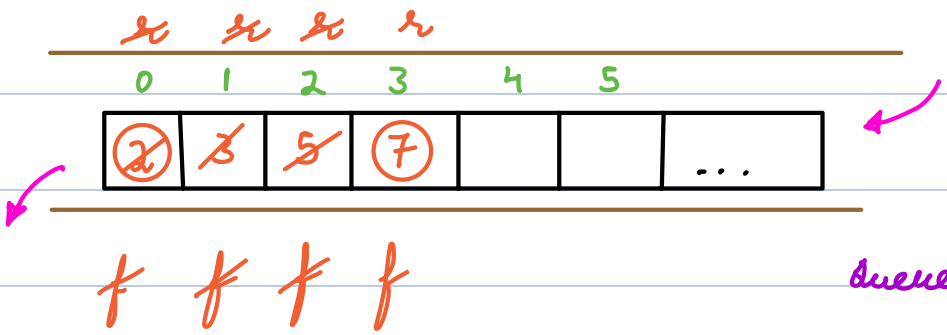
TC = O(1)

Q1: Implement Queue using Arrays

Example: enqueue(2) enqueue(3) enqueue(5) front( ) → 2 dequeue( )

dequeue( ) isEmpty( ) → false enqueue(7) rear( ) → 7 dequeue( ) front( ) → 7





queue A[f _ r]

$r = -1$ $f = 0$

```
boolean isEmpty() {
    return (r - f + 1) == 0   // r < f
}
```

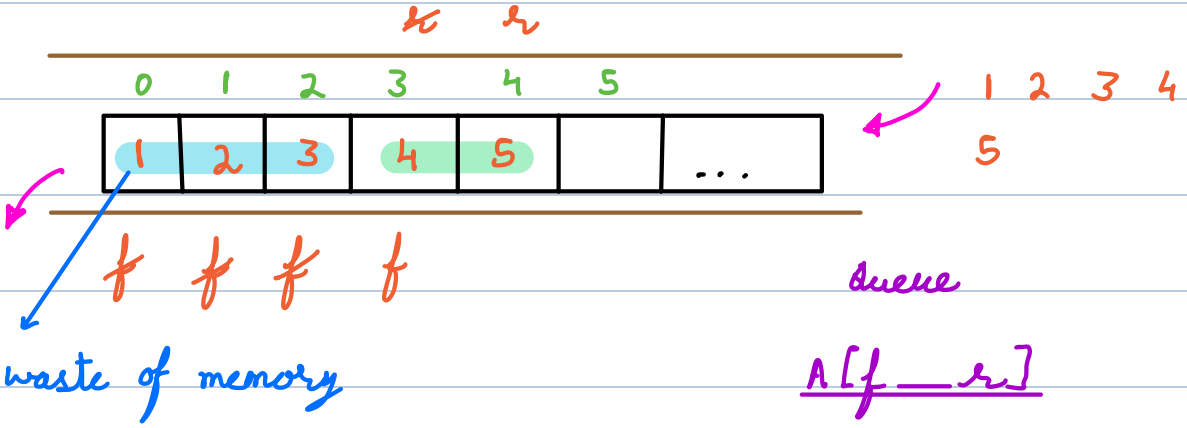
```
void enqueue(x) {   // overflow → using dynamic array
    r++
    A[r] = x
}
```

```
int dequeue () {        // underflow
    if (isEmpty())
        return -1
    f++
    return A[f-1]
}
```

TC = O(1) ∀ functions

```
int front () {
    if (isEmpty())
        return -1
    return A[f]
}
```

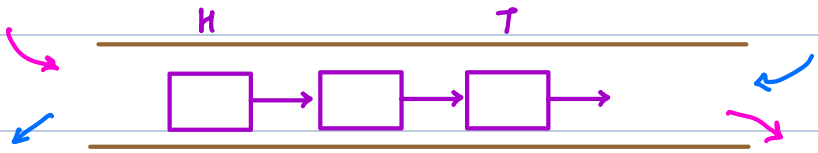
```
int rear () {
    if (isEmpty())
        return -1
    return A[r]
}
```



Q2: Implementation of Queue using Linked List

Example: enqueue(2) enqueue(3) enqueue(5) front( ) dequeue( )

dequeue( ) isEmpty( ) enqueue(7) rear( ) dequeue( ) front( )



enqueue (x) → insert as head

dequeue () → remove tail

TC = O(N) TC = O(1)

enqueue (x) → insert at tail

dequeue () → remove head

TC = O(1) TC = O(1)

Head = null Tail = null

```
boolean isEmpty () {
    return Head == null
}
```

```
void enqueue (x) {
    nr = new Node (x)
    if (isEmpty ()) { Head = nr
                      Tail = nr }
    else { Tail.next = nr
           Tail = nr }
}
```

```
int dequeue () {
    if (isEmpty ()) return -1
    x = Head.val
    Head = Head.next
    if (Head == null) Tail = null
    return x }
```

TC = O(1) ∀ functions

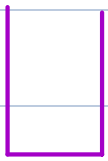
```
int front () {
    if (isEmpty ())
        return -1
    return Head.val
}
```

```
int rear () {
    if (isEmpty ())
        return -1
    return Tail.val
}
```

Q3: Implementation of Queue using 2 Stacks

Example: enqueue(2) enqueue(3) enqueue(5) front( ) dequeue( )

dequeue( ) isEmpty( ) enqueue(7) rear( ) dequeue( ) front( )

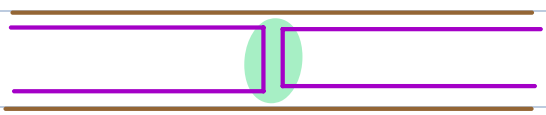


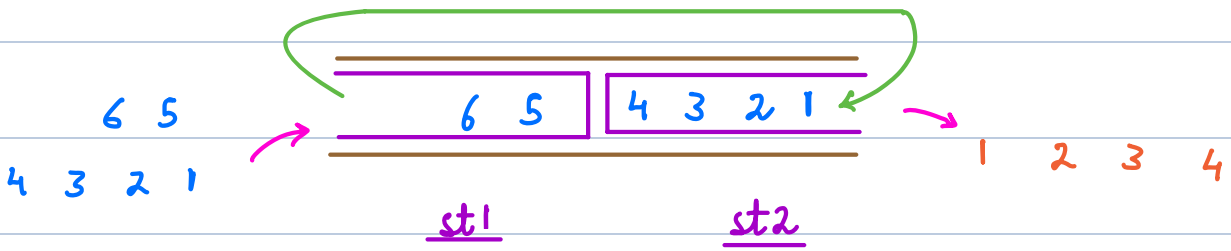
stacks → push(x) pop( ) peek( ) isEmpty( )

enqueue(x)

dequeue( )

isEmpty( )






```
boolean isEmpty() {
    return (st1.isEmpty() && st2.isEmpty())
}
```

```
void enqueue(x) {
    st1.push(x)
}
```

```
void move() {
    while(!st1.isEmpty())
        st2.push(st1.pop())
}
```

```
int dequeue() {
    if(!isEmpty()) return -1
    if(st2.isEmpty()) move()
    return st2.pop()
}
```

If K elements are moved from S1 to S2 → TC = O(K)

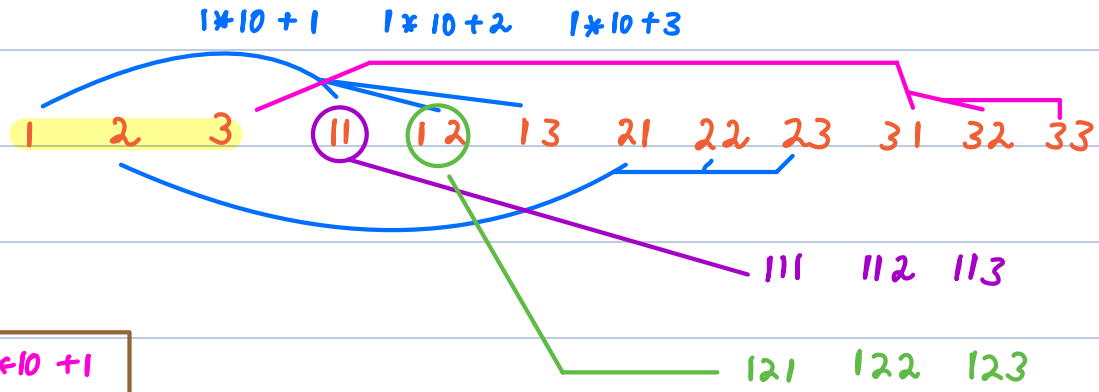
then for K dequeue operations → TC = O(1) per operation

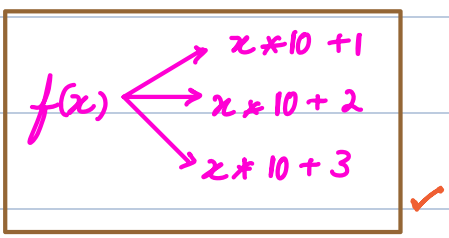
⇒ TC per element = O(2) → O(1) for dequeue

Q4: Given an integer **A**, generate the first **A** positive integers that contain only digits **1**, **2**, and **3**.

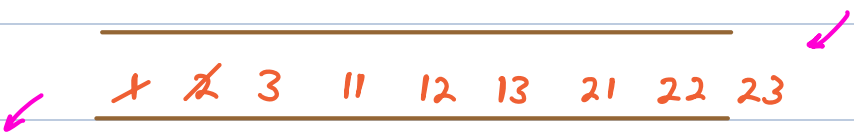
Example: A = 13 {1, 2, 3, 11, 12, 13, 21, 22, 23, 31, 32, 33, 111}

1 2 3 4 5 6 7 8 9 10 11 12 13





FIFO → queues



if (A <= 3) return A

TC = O(N) SC = O(N)

queue

cnt = 3 6 9

Consider the following queue implementation using a dynamic array:

```
enqueue(x):
    arr[rear++] = x

dequeue():
    return arr[front++]
```

overflow

Assume the array size is 100, initially front = 0, rear = 0. After 100 enqueue operations followed by 100 dequeue operations, what happens if we try to enqueue again?

Consider the lazy transfer approach for implementing a queue using two stacks.

```
enqueue(x):
    push(stack1, x)

dequeue():
    if stack2 is empty
        move all elements from stack1 to stack2
    pop(stack2)
```

If we perform n enqueue operations followed by n dequeue operations, what is the amortized time complexity per operation?

TC = O(1)

A queue is used to generate numbers using digits 1, 2, and 3 in increasing order. Algorithm:

```
1. Insert "1", "2", "3" into queue

2. Repeat:

 - Remove front element x
 - Print x
 - Insert x+"1", x+"2", x+"3"
```

What is the 7th number printed?

1 2 3 11 12 13 (21)